

static-локална променлива → запомня стойността между извикванията
-глобална/функция → видима само в този .cpp файл
-член на клас → споделен между всички обекти на класа
Тоест: Само един екземпляр за целия живот на програмата.
static

Памет в C++:

- Стек - тук са локалните променливи
- Heap - тук са динамично зарезервираните обекти
- Data segment - тук живеят глобалните и static променливи

static локална променлива - инициализира се само веднъж

static глобална променлива - видима е само в текущия .cpp файл, което предотвратява конфликти на имена

static функция в .cpp файл - видима само в текущия .cpp файл; при вътрешни помощни функции, които не трябва да са достъпни от други файлове

static член-променлива на клас - едно копие за всички обекти

- декларира се в тялото на класа
- дефинира се в .cpp файл - там се зарежда памет и се инициализира

! static const > инициализират се директно в тялото на класа
• static constexpr

static метод на клас - принадлежи на класа, не на конкретен обект
-извиква се без да е създаден обект
-няма this указател
-не може да достъпва нестатични членове

Дизайн патърн Singleton

Дизайн патърн - концепция за решение на често срещан проблем;
начин на мислене за структурирането на кода

Singleton - клас, от който може да съществува точно един обект за целия живот на програмата

• Използва се, когато ни трябва точно едно копие:

- връзка към база данни
- системен журнал (logger)
- конфигурация на приложение
- мениджър на ресурси

Четири задължителни елемента на Singleton:

```
class Singleton {  
public: 1. Публичен статик метод за достъп  
    static Singleton& getInstance();  
    static Singleton instance; 2. static локална  
    return instance;           конструира се веднъж  
}  
3. Забраняваме копирането  
Singleton (const Singleton& other) = delete;  
Singleton& operator= (const Singleton& other) = delete;  
private:  
    Singleton() {} 4. Конструкторът е private  
}
```

⊕ Гарантира точно 1 обект
Глобален достъп без глобална
променлива
Лесна имплементация

⊖ Трудно се тества
Живее до края на програмата
Функциите зависят от него без да е
деклариран явно